01 COMP213 · Operating Systems

Computer System & OS Overview

Foundational Journey Through Hardware & OS Principles



August , 2026

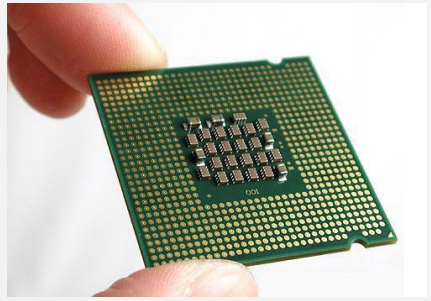
Xing Zheng / Dylan

What is a Computer System?

A computer system has four core structural elements that work together

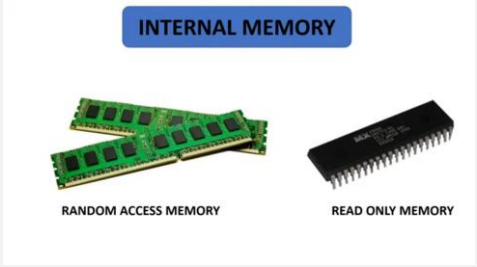
Processor

Controls operations and processes data (your CPU)



Memory

Stores programs and data temporarily while running



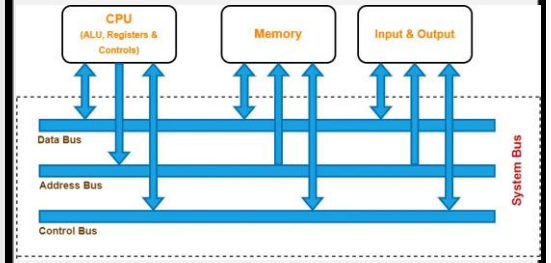
I/O Modules

Connects to external devices: keyboards, disks, networks



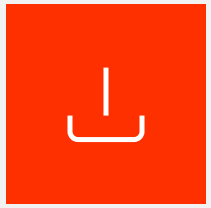
System Bus

The communication highway linking all components



The Basic Instruction Cycle

Every program you run follows a simple but powerful loop



Fetch

Your CPU grabs the next instruction from memory

Uses the **Program Counter (PC)**

Then



Execute

The CPU decodes and runs that instruction

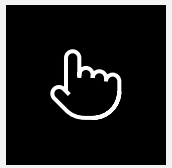
Move data • Calculate • Change flow



This cycle repeats **billions of times per second** — keeping your computer alive and responsive!

Inside the CPU Registers

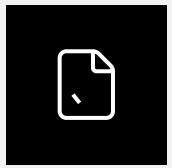
Let's look at three essential registers you'll work with



PC

Program Counter

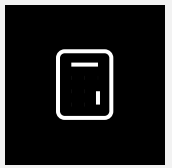
Holds the **address of the next instruction** to fetch from memory



IR

Instruction Register

Stores the **current instruction** being executed by the CPU



AC

Accumulator

Temporary storage for arithmetic operations and data

A Program Execution Walkthrough

Tracing how the CPU adds two values from memory: a step-by-step demonstration

01

Fetch Stage

CPU retrieves instruction from address stored in **Program Counter (PC)**

02

Execute Stage

Performs operation (load, add, store) using **Accumulator (AC)** as working register

03

Store Result

Final sum written back: memory[940] + memory[941] → **memory[941]**



Why Interrupts Matter

Problem

Without Interrupts

The CPU **idles**—wasting millions of cycles—while waiting for slow devices like printers or disks to finish

Solution

With Interrupts

The CPU **delegates I/O tasks** and immediately resumes useful work

When the device finishes, it **interrupts** the CPU to signal completion

What would happen if your laptop **froze** every time you saved a file?

The Interrupt-Driven I/O Flow

Let's compare how your CPU handles I/O in two different ways

Program-Controlled I/O

CPU waits idly during slow device operations

→ Wastes CPU cycles

VS

Interrupt-Driven I/O

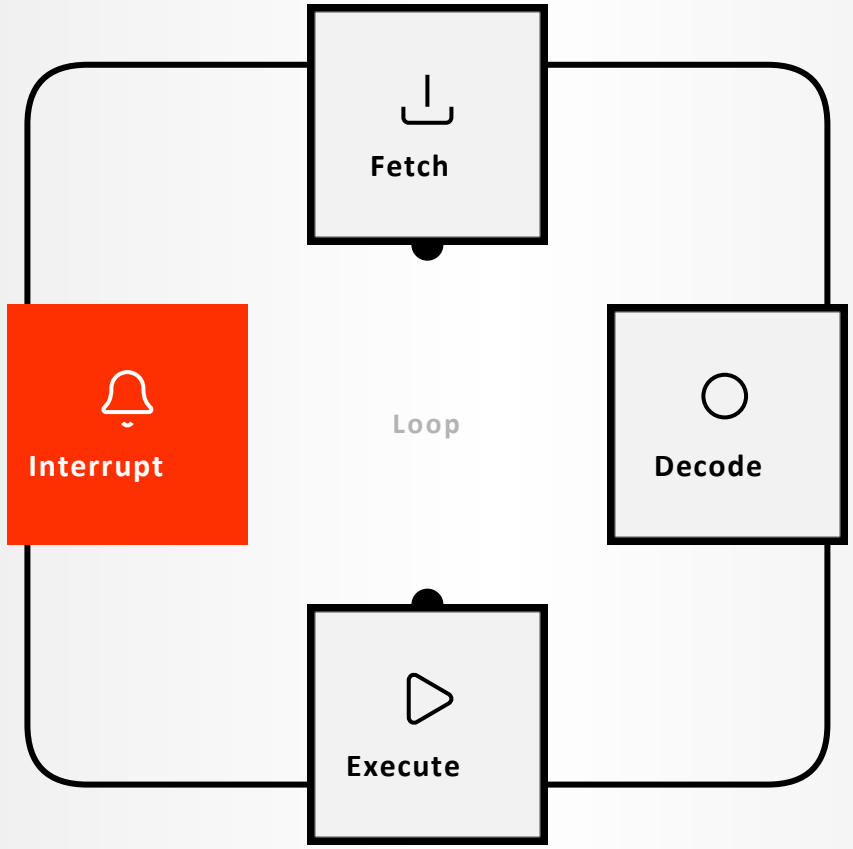
CPU continues useful work while device operates

→ Resumes when device signals completion

Which approach keeps your system more efficient?

The Enhanced Instruction Cycle

Now we add an **interrupt stage** to the basic fetch-execute cycle



What's New

After each **execute phase**, the processor checks for pending interrupts before fetching the next instruction

Why It Matters

This simple addition lets your computer handle **urgent events** (keyboard presses, disk completion) without wasting cycles waiting

Think About It

Why do you think the interrupt check happens *after* execute but *before* the next fetch?

Types of Interrupts

Four fundamental interrupt types that CPU handles during execution

Program

Triggered by your code—like division by zero, illegal instructions, or accessing memory you shouldn't

Timer

Sent by the CPU's internal clock so the OS can run tasks regularly (e.g., scheduling)

I/O

Raised by devices to say "I'm done!" or "Something went wrong!" after reading/writing data

Hardware Failure

Caused by serious issues like power loss or memory errors—rare but critical

Handling Multiple Interrupts

When multiple interrupts occur simultaneously, systems must decide how to manage them



Disable Interrupts

Temporarily ignore new interrupts while handling the current one

- Simple implementation
- May miss time-critical events



Interrupt Priority

Allow higher-priority interrupts to preempt lower-priority handlers

- More responsive to urgent events
- Adds system complexity



Which approach would you choose for a **medical monitoring system** versus a **basic calculator**?

Ch1 Summary: Hardware Basics

Three fundamental concepts you must understand about computer hardware organization

01

Four Core Components

A computer has four core parts: **processor** (executes instructions), **memory** (stores data/programs), **I/O modules** (connect to external devices), and the **system bus** (links everything)

02

Fetch-Execute Cycle

Instructions run in a **fetch-execute cycle**: the CPU fetches an instruction (using the **Program Counter**), decodes it (**Instruction Register**), then executes it—often updating data in registers like the **Accumulator**

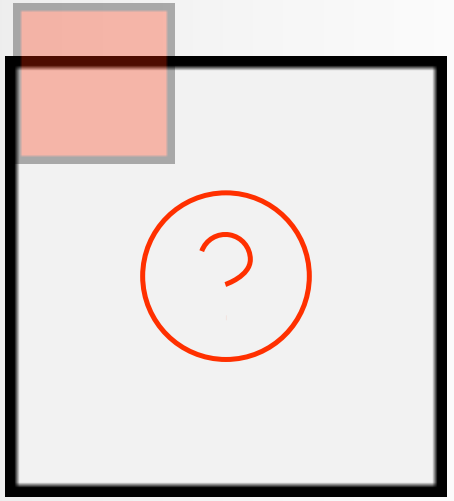
03

Interrupts

Interrupts let slow I/O devices signal the CPU without wasting cycles; the processor pauses, runs an **interrupt handler**, then resumes—keeping the system efficient and responsive

Think About It: No Interrupts?

What would happen to system performance and responsiveness without interrupts?



- Your CPU would waste **thousands of cycles** waiting for slow devices like printers or disks
- The whole system would feel **unresponsive**—like your mouse or keyboard freezing during file transfers

How would YOU design a better solution?

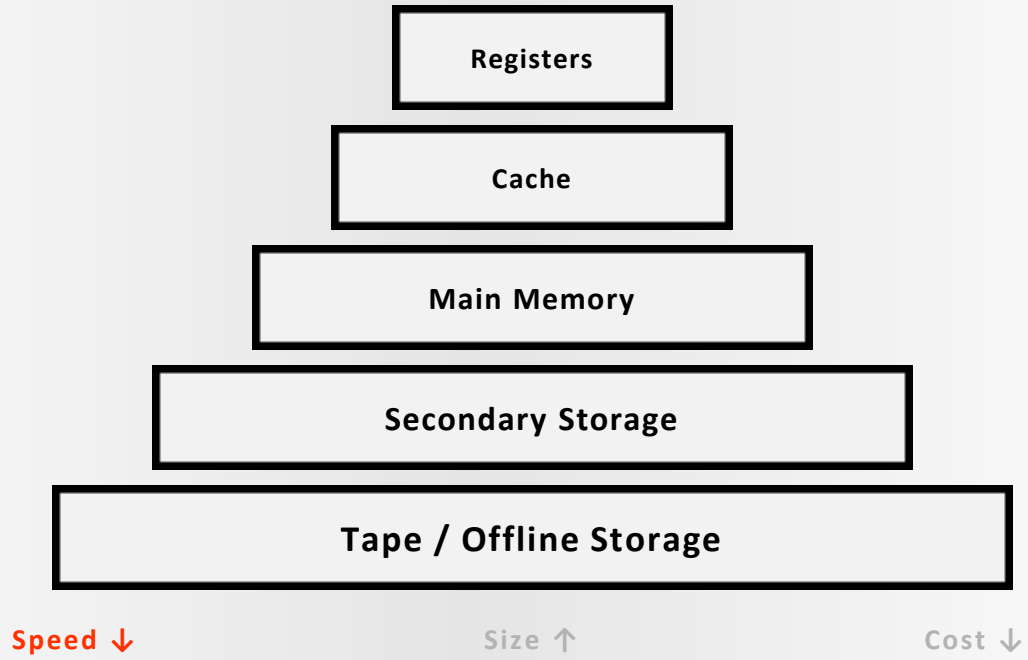
The Memory Hierarchy Concept

Memory Pyramid Structure

Fastest, smallest, most expensive (registers, cache) at the top → **slowest, largest, cheapest** (disk, tape) at the bottom

The Trade-off

As **speed ↑**, **size ↓** and **cost per byte ↑**. You must balance these factors



Why can't we just use the fastest memory everywhere?

Why Caches Exist

Principle of Locality

Programs tend to access the same or nearby memory locations repeatedly

Temporal Locality

Accessing the **same memory locations** repeatedly over time

Recent past → Near future

Spatial Locality

Accessing **nearby memory locations** in close succession

Close addresses → Sequential access

How Cache Memory Works



Mapping

How main memory blocks are placed in cache
Direct · Associative · Set-associative



Replacement

Which block to evict when cache is full
LRU · FIFO · Random



Write Policy

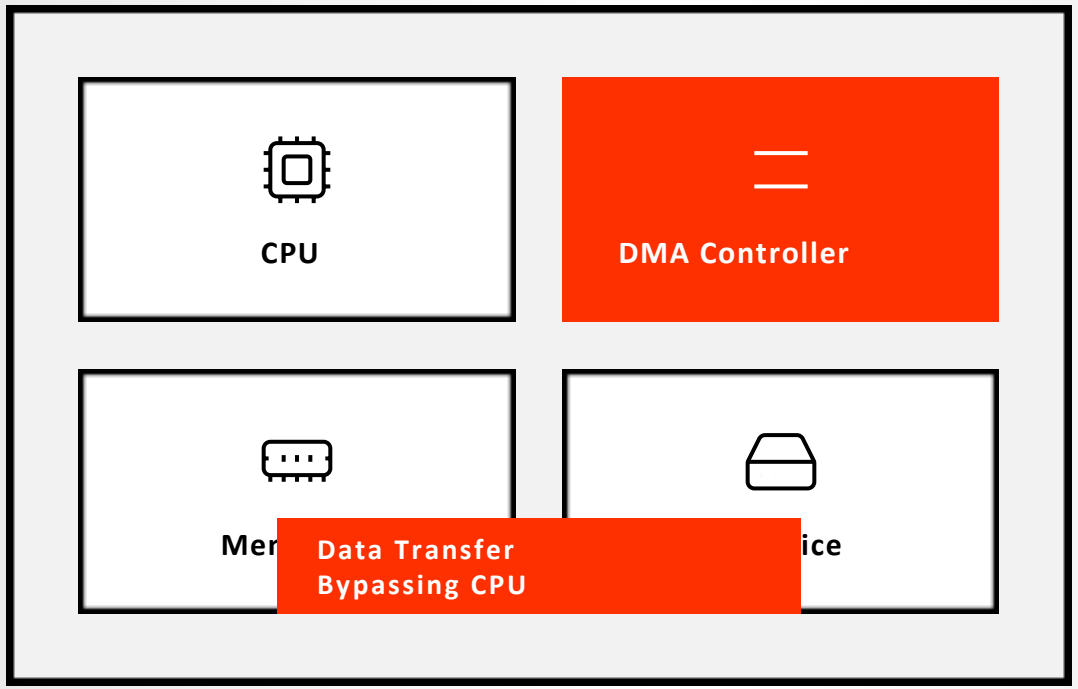
How writes are handled to maintain consistency
Write-through · Write-back



Which policy combination would work best for a mobile device with limited battery?

3

Direct Memory Access (DMA)



What it is

A dedicated DMA controller handles data transfers between I/O devices and memory—**bypassing the CPU** entirely

Why it matters

Frees the CPU to do useful work instead of waiting during slow, bulk transfers (e.g., disk reads)

Key benefit

Boosts system efficiency by **reducing CPU overhead** for large data movements

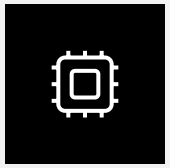
Think About It

What would happen if the CPU had to copy every byte from a hard drive?

From Single Core to Multicore

This evolution enables true parallel processing capabilities

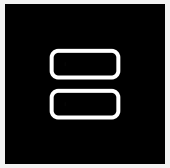
01



Single-Core

One processor handling all tasks sequentially in early systems

02



SMP

Multiple processors sharing memory, managed by one OS

03



Multicore

Multiple cores on a single chip, each acting independently

Discussion Question

How might software need to change to take advantage of multiple cores?

3

Recap & Discussion Questions

01

Memory Hierarchy & Cache

Faster, smaller caches exploit **locality** to bridge the CPU-memory speed gap

Why can't we just use all cache instead of DRAM?

02

DMA & Multicore

DMA frees CPU from I/O data copying; **Multicore** chips enable parallel threads

How might DMA and multicore features interact in a real system?

What is an Operating System?

Two fundamental roles that define every OS



Resource Manager

Allocates **CPU time**, **memory**, **disk space**, and **I/O devices** fairly and efficiently among competing programs



Control Program

Supervises execution of user programs to **prevent errors** and manage hardware access safely

Think About It



What would happen if applications could directly control hardware without an OS?

2

Core OS Objectives

Every operating system aims to balance three key goals

01  **Convenience**

Making the computer easy to use through clean abstractions and intuitive interfaces

02  **Efficiency**

Maximizing resource utilization (CPU, memory, I/O) without waste

03  **Evolvability**

Allowing the system to adapt and grow with new hardware and software needs



Think About It

Which of these goals matters most for your smartphone's OS?

Essential OS Functions

The OS handles six core jobs to keep everything running smoothly

Process Management

Creates, schedules, and terminates running programs

Memory Management

Allocates RAM fairly and keeps processes isolated

File Management

Organizes, stores, and retrieves your data reliably

I/O Management

Controls communication with devices like disks and keyboards

Protection

Enforces security rules so programs don't interfere with each other

User Interface

Provides shells or GUIs so you can actually use the system

The OS as a Service Provider

Think of the operating system as your program's personal assistant — your code simply requests services from the OS

Process Management

Creating and scheduling program execution

Memory Allocation

Providing safe, isolated memory space

File Operations

Reading/writing data without knowing disk details

I/O Handling

Managing printers, keyboards, and network devices

What would happen if every application had to manage hardware directly?

Early OS: Serial Processing

01

No Operating System

Programmers worked directly with bare metal hardware — no software layer in between

02

Sequential Execution

Each program ran one after another: load → execute → unload before the next job

03

Manual Operation

You had to manually set switches or punch cards for every single instruction

What challenges would you face if you had to operate a computer this way today?

Simple Batch Systems

01

Batch Processing

Groups similar jobs together to reduce setup time between tasks

02

The Monitor

Early OS automatically loads and runs the next job in sequence

03

Improved Efficiency

Eliminates manual intervention between jobs, improving CPU utilization

Think About It

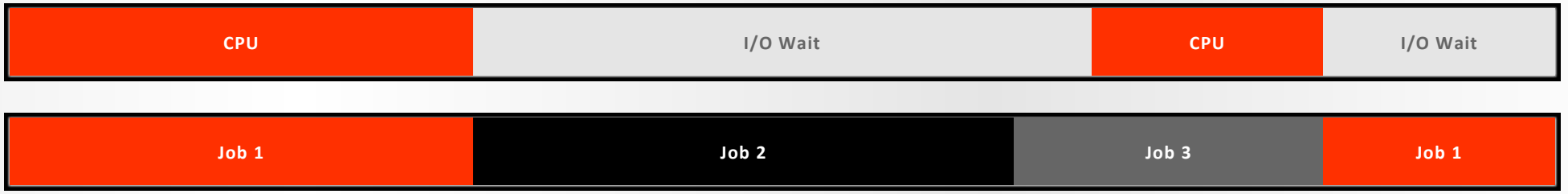
How would you design a system to handle different types of jobs fairly?

Multiprogramming Revolution

Multiprogramming keeps several jobs in memory at once, so when one job waits for I/O, the CPU can immediately switch to another ready job

Dramatically boosts CPU utilization

CPU Utilization Comparison



← CPU stays busy! No more idle time →

Think About It

What happens if we only run one job at a time? How much CPU time goes to waste?

Time-Sharing & Multitasking

Core Concept

Time-sharing lets multiple users interact with a computer simultaneously by rapidly switching the CPU between their jobs

1. Each user gets the **illusion** of having exclusive machine access
2. Switching happens so fast (milliseconds!) that you won't notice delays
3. [redacted] batch processing to enable **interactive**

Think About It

What would happen if the OS didn't switch tasks quickly enough? How slow is *"too slow"* for you to feel it's unresponsive?

Modern OS Milestones

01 Microkernels

Move core services out of kernel space — improves **modularity** and **security**

02 Multithreading

Let a single process run multiple **threads** — boosts responsiveness & CPU utilization

03 SMP Support

Manage **Symmetric Multiprocessors** — multiple CPUs sharing memory for scalability

04 Object-Oriented Design

Structure OS components as **reusable objects** — eases maintenance & extension

Think About It

How might these ideas shape the OS you'll build?

Summary: OS Evolution

01

Serial Processing

No OS: Programmers ran one job at a time directly on hardware

02

Multiprogramming & Time-sharing

OS emerged to manage resources, enable concurrency, and support interactive use

Modern OS Foundation

Critical Thinking

Each step solved real limitations. What if we skipped multiprogramming—could modern apps even exist?

Imagine This:

No OS?

What if you had to **manually**:

- Load programs into memory
- Switch between applications
- Manage CPU time fairly
- Handle I/O operations directly

How would you keep everything running smoothly—without crashing?

Major OS Achievements

Modern operating systems solve three fundamental challenges that define computing



01

Process Management

Creating, scheduling, and coordinating multiple programs so they run smoothly together



02

Memory Management

Giving each program its own "private" memory space while efficiently sharing physical RAM



03

Information Protection

Controlling who can access what resources to keep the system secure and stable

Process Management Basics

The operating system treats each program as a process—a unit of work that it manages from start to finish

Creates

Starts a new process when you launch a program

Schedules

Decides which process runs on the CPU and when

Terminates

Ends the process cleanly when it's done or crashes

Think About It

What might happen if the OS couldn't terminate a stuck process?

Memory Management Goals

Your OS needs to balance three key goals when managing memory

Isolation

Keep each process's memory separate so bugs or attacks in one program can't affect others

Sharing

Allow processes to safely share memory when needed (like shared libraries)

Efficient Usage

Make the most of limited RAM by using techniques like **virtual memory**

Discussion Question

How would you design a system that achieves all three goals at once?

Virtual Memory Concept

What Process Sees

Virtual Address Space

Each process believes it has its own huge, private memory

OS Maps Behind the Scenes

Physical Reality

Physical Memory (RAM)

Limited hardware memory shared by all processes

Address Space Illusion

Each process gets its own huge, private address space —even when physical RAM is limited

Address Translation

OS maps virtual addresses (used by programs) to physical addresses (in real memory)

Process Isolation

Every process gains isolation and the feeling of abundant memory

How would programs behave if they shared one global address space?

Information Protection & Security

Access Control

Mechanisms decide who can use what resources

Protected Space

Each process runs in isolation – you can't mess with others' data

Permissions

The OS enforces read, write, execute – like digital locks on files

How would your phone feel if any app could read your messages or delete photos?

Scheduling Fundamentals

The OS uses a scheduler to decide which ready process gets the CPU next — like a traffic cop managing flow through an intersection

1 The Scheduler

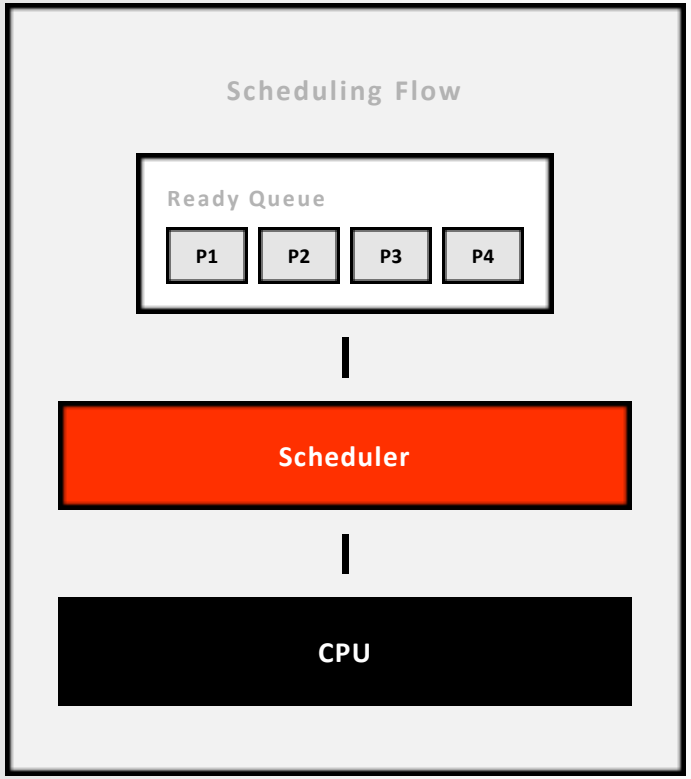
A critical OS component that selects which process in the ready queue gets CPU time next

2 Ready Queue

Maintains all processes waiting for CPU — without scheduling, only one program runs at a time

3 Scheduling Algorithms

Apply fair and efficient decision-making rules to allocate processor time among competing processes



Critical Thinking

How would you design a fair system if you had to choose between urgent tasks and long-running computations?

System Generation (SYSGEN)

What is SYSGEN?

System Generation is the process of customizing and configuring an operating system for a specific computer's hardware setup

When does SYSGEN happen?

01 Installing an OS on new hardware

02 The OS needs to recognize your exact CPU, memory, and devices

03 Drivers and kernel modules are selected based on your system

Think About It

What might go wrong if an OS skips proper SYSGEN for your laptop?

Key Concepts

Four pillars that define modern operating systems

Processes

The OS manages programs as **processes**—isolated units of execution with their own state and resources

Memory

Virtual memory gives each process the illusion of a large, private address space, even on limited physical RAM

Protection

Hardware and OS mechanisms enforce **access control**, ensuring processes can't interfere with each other or the kernel

Scheduling

The **scheduler** decides which ready process runs next, balancing fairness, responsiveness, and throughput

How do these **four pillars** work together to make multitasking feel seamless?

Puzzle This: Resource Sharing

How can the OS safely allow multiple processes to share the same printer?

01 Should processes send print jobs directly to the printer?

02 What could go wrong if two processes print simultaneously?

03 Could we use a [redacted] to manage print requests?

04 What role does the OS play as a [redacted] here?

OS for Multiprocessor Systems

In **SMP** (Symmetric Multiprocessor) and multicore systems, the OS faces three key challenges

Scheduling

The OS must distribute processes across multiple CPUs fairly and efficiently

Synchronization

Shared resources need careful coordination to avoid race conditions

Memory Management

Ensures cache coherency and efficient memory access across cores

How would you design a scheduler that balances load while minimizing inter-core communication overhead?

3

Modern OS Characteristics

Three key architectural innovations that define modern operating systems



Microkernel Architecture

Keeps the kernel minimal—**only essential services** (like IPC) run in kernel mode. Everything else (drivers, file systems) runs as user-space processes for better **stability and security**



Multithreading

Lets a single process have **multiple threads of execution**, improving responsiveness and enabling efficient use of **multicore CPUs**



Object-Oriented Design

Treats system resources (files, devices, processes) as **objects with defined interfaces**—making the OS more modular, reusable, and easier to extend

Final Recap: From Hardware to OS

The operating system transforms raw hardware complexity into usable abstractions

01



Raw Hardware

Your computer's ~~hardware~~ (CPU, memory, I/O) is raw and complex—but the OS hides this complexity through **abstraction**

02



OS as Bridge

Think of the OS as a bridge: it turns physical resources into easy-to-use services like **processes**, **virtual memory**, and **files**

03



Why It Matters

Without these abstractions, you'd manage every register, interrupt, and memory address yourself—imagine coding **that**!

Let's Discuss & Explore!

Open questions for further exploration and critical thinking

Question 01



What if we redesigned interrupts for today's **multicore systems**—how would you approach it?

Question 02



Where do you think operating systems will **evolve next** as hardware keeps changing?

Question 03



If you were building an OS from scratch today, what would be your **top priority**?

千问AI生成